ASSIGNMENT-13.5

Name : Nishitha

Roll no : 2303A51540

B19

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

Week7 -

Friday

• Identify code smells and inefficiencies in legacy Python scripts.

• Use AI-assisted coding tools to refactor for readability, maintainability, and performance.

• Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):
```

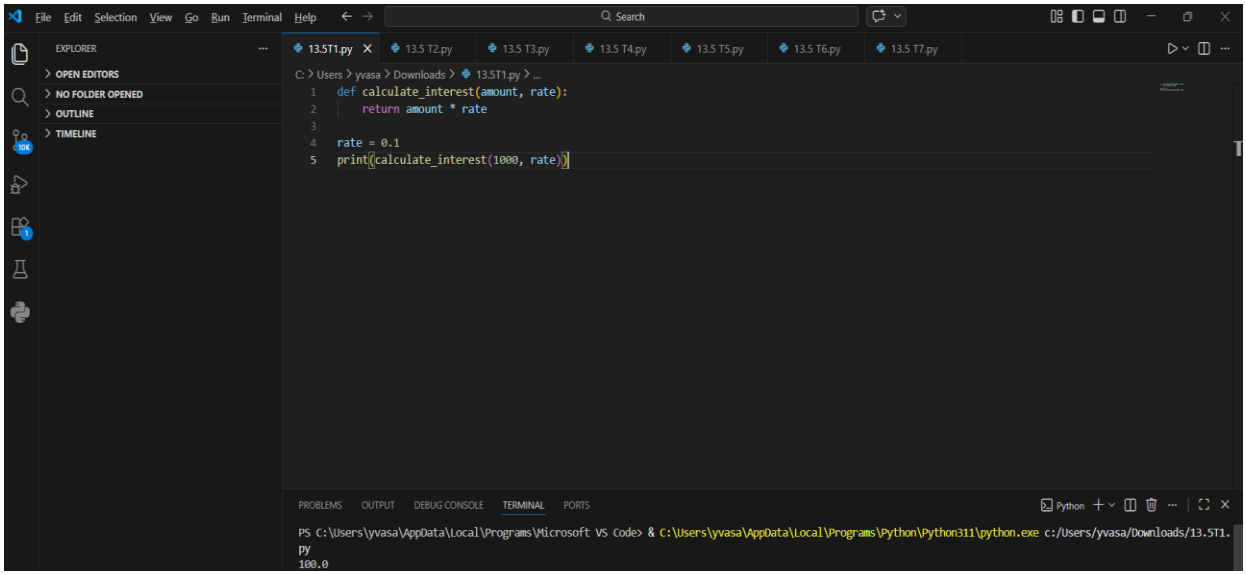
return amount * rate

print(calculate_interest(1000))

• Expected Output:

o Refactored version passing rate as a parameter or using a

configuration structure.



Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a

cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

print("Very Good")

else:

if score >= 60:

print("Good")

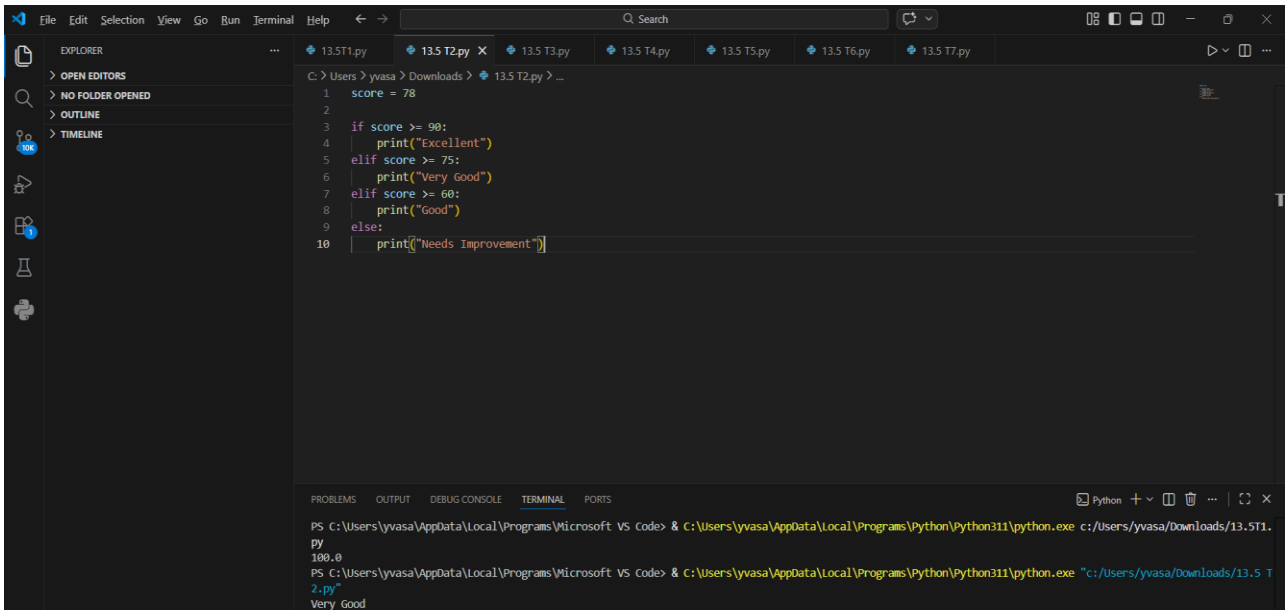
else:

print("Needs Improvement")

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based

approach.



Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

f = open("data1.txt")

print(f.read())

f.close()

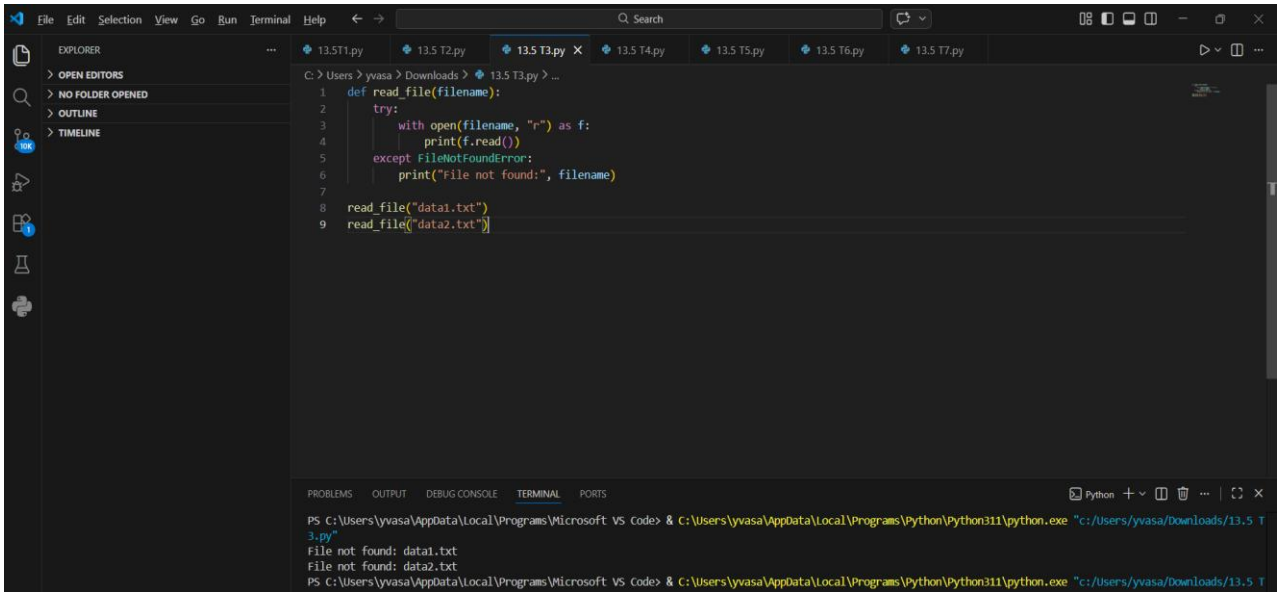
f = open("data2.txt")

print(f.read())

f.close()

Expected Outcome:

o Reusable function using with open() and parameters.



Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate

data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

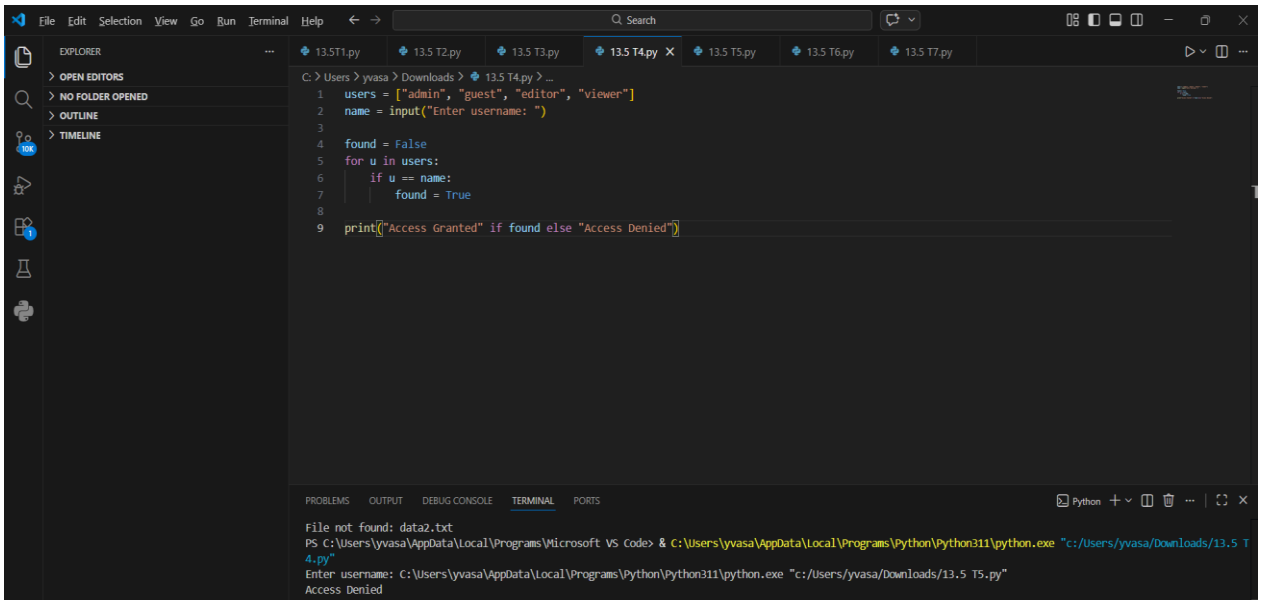
if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.



Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based

design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and

attributes.

```python
class EmployeeSalaryCalculator:

    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def net_salary(self):
        return self.salary - self.calculate_tax()


employee = EmployeeSalaryCalculator(50000)
print(employee.net_salary())
```

```
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T5.py"
40000.0
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code>
```

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling

large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

if i % 2 == 0:

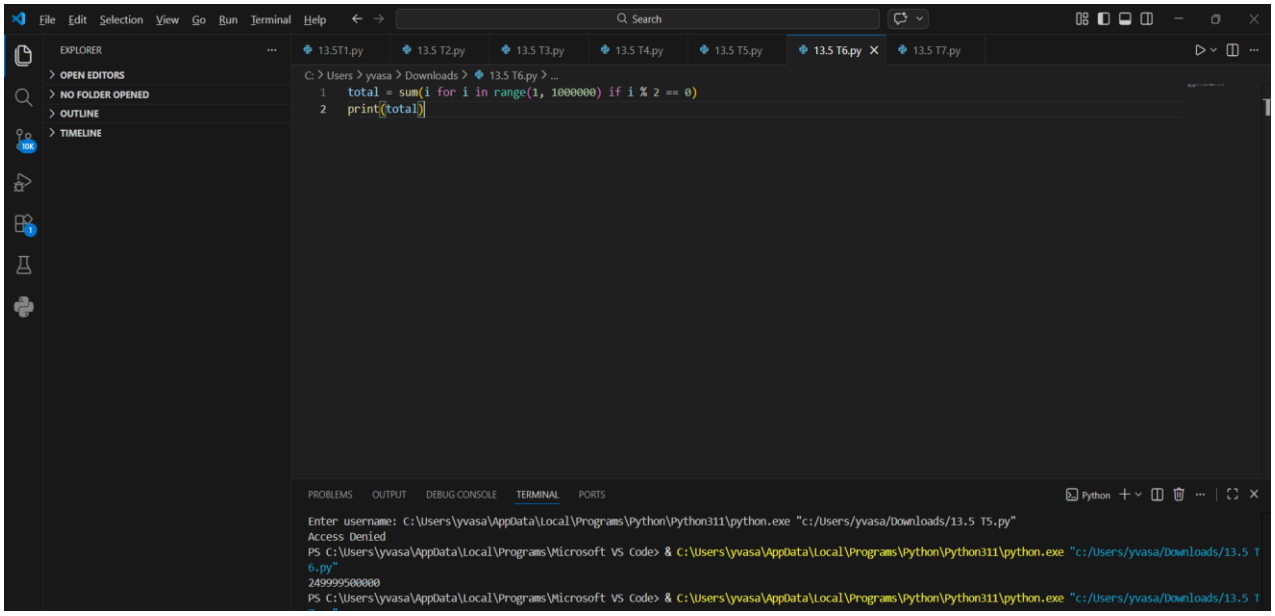
total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or

comprehensions, with time comparison.



Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

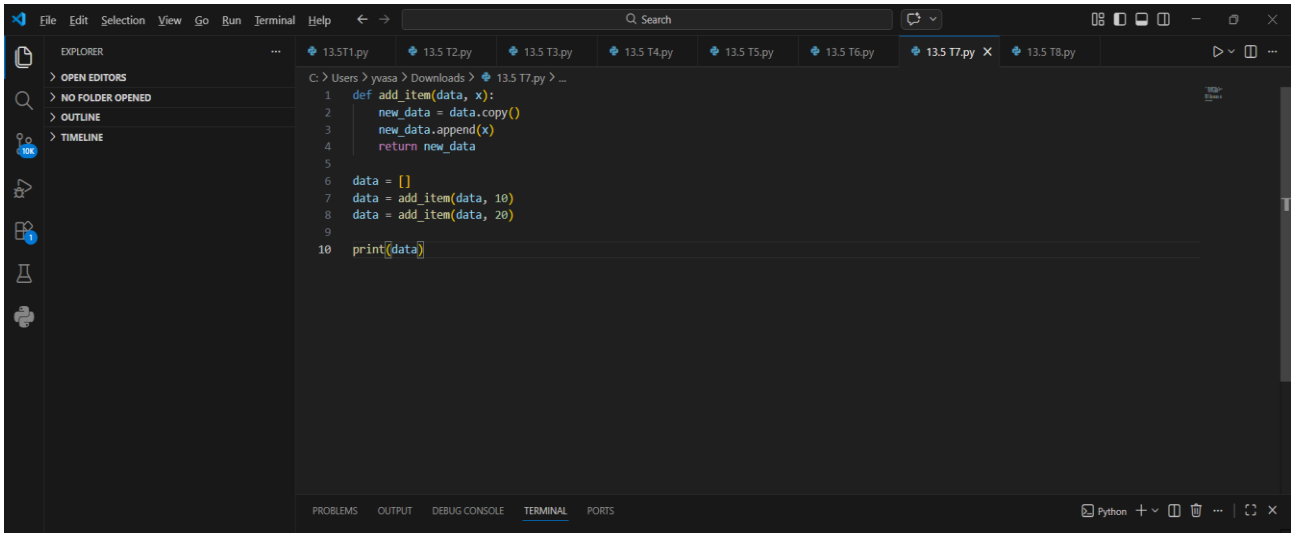
Legacy Code:

```
data = []

def add_item(x):

data.append(x)

add_item(10)

add_item(20)

print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating

globals.




```
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T
7.py"
[10, 20]
```

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation

rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

print("Must contain uppercase")

else:

print("Must contain digit")

else:

print("Password too short")

Expected Outcome:

o Separate validation functions with clear responsibility

```python
def has_min_length(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)


password = input("Enter password: ")

if not has_min_length(password):
    print("Password too short")
elif not has_digit(password):
    print("Must contain digit")
elif not has_uppercase(password):
    print("Must contain uppercase")
else:
    print("Valid Password")
```

```
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T
8.py"
Enter password: 123456
Password too short
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code>
```